

Deadlock ինչ է — Խորը ուղեցույց

C++ Concurrency presentation

Այս ուղեցույցը մանրամասն բացատրում է deadlock-ը՝ ինչ է, 4 պայմանը, իրական օրինակ (սխալ + ճիշտ լուծում), ինչպես խուսափել, և հարցազրույցի հարցեր:

1. Ի՞նչ է Deadlock-ը

Deadlock-ը լինում է, երբ **երկու (կամ ավելի) thread սպասում են իրար** անվերջորեն, ու ոչ մեկը չի կարողանում առաջ գնալ:

```
Thread A: պահում է Lock1, սպասում է Lock2
Thread B: պահում է Lock2, սպասում է Lock1
-> Երկուսն էլ սպասում են -> ԿԱԽՎԱԾ (deadlock)
```

2. Իրական օրինակ (ՍԽԱԼ կոդ)

```
std::mutex m1, m2;

void threadA() {
    std::lock_guard<std::mutex> l1(m1);    // պահում m1
    std::lock_guard<std::mutex> l2(m2);    // սպասում m2
}

void threadB() {
    std::lock_guard<std::mutex> l1(m2);    // պահում m2
    std::lock_guard<std::mutex> l2(m1);    // սպասում m1
}

// A պահում m1 սպասում m2; B պահում m2 սպասում m1 -> DEADLOCK
```

Խնդիրը՝ **տարբեր կարգով** են lock անում (A: m1->m2, B: m2->m1):

3. Deadlock-ի 4 պայմանը (Coffman conditions)

Deadlock լինելու համար պետք են **բոլոր 4-ը** միասին:

1. Mutual Exclusion -> resource-ը միայն մեկ thread միանգամից
2. Hold and Wait -> thread պահում է մեկ, սպասում մյուսին
3. No Preemption -> resource-ը բռնով չի արվում (կամովին վերադարձնում)
4. Circular Wait -> A սպասում B-ին, B սպասում A-ին (շրջան)

Մեկը կոտրես -> deadlock չի լինի:

4. Լուծում 1 — Նույն կարգով lock

```
void threadA() {
    std::lock_guard<std::mutex> l1(m1);    // m1 առաջ
    std::lock_guard<std::mutex> l2(m2);    // հետո m2
}

void threadB() {
    std::lock_guard<std::mutex> l1(m1);    // ՆՈՒՅՆ կարգ: m1 առաջ
    std::lock_guard<std::mutex> l2(m2);    // հետո m2
}

// Երկուսն էլ նույն կարգով -> circular wait կոտրվում -> ՉԿԱ deadlock
```

Ամենապարզ լուծումը. **միշտ նույն կարգով lock անի mutex-ները:**

5. ԼՈՒԾՈՒՄ 2 — std::lock (atomic մի քանի lock)

```
void threadA() {
    std::lock(m1, m2);    // երկուսն էլ միասին lock (deadlock-free)
    std::lock_guard<std::mutex> l1(m1, std::adopt_lock);
    std::lock_guard<std::mutex> l2(m2, std::adopt_lock);
}
```

`std::lock` -ը երկու mutex-ը lock է անում atomic (deadlock-ից ազատ ալգորիթմով): C++17-ում՝ `std::scoped_lock`:

```
std::scoped_lock lock(m1, m2);    // C++17 -- մի քանի mutex, deadlock-free
```

6. ԼՈՒԾՈՒՄ 3 — try_lock (timeout)

```
std::timed_mutex m;
if (m.try_lock_for(std::chrono::seconds(1))) {
    // աշխատանք
    m.unlock();
} else {
    // չստացավ -> այլ բանով շրջանցի (deadlock-ից խուսափել)
}
```

7. ԽՈՒՍԱՓԵԼՈՒ ձևերը — ամփոփ

1. Միշտ ՆՈՒՅՆ ԿԱՐԳՈՎ lock և mutex-ները (ամենապարզ)
2. `std::lock` / `std::scoped_lock` (atomic մի քանի lock)
3. `try_lock` timeout-ով
4. Քիչ mutex, փոքր critical section
5. Չպահել lock երկար (I/O lock-ի մեջ չէ)

8. Հարցազրույցի հարցեր և պատասխաններ

Deadlock ի՞նչ է:

Երկու+ thread սպասում են իրար անվերջորեն; ոչ մեկը առաջ չի գնում (կախված):

Ինչո՞ւ է լինում:

Thread-ներ տարբեր կարգով մի քանի lock -> circular wait:

4 պայմանները:

Mutual exclusion, hold and wait, no preemption, circular wait (Coffman). Բոլոր 4-ը պետք են:

Ինչպես լուծել/խուսափել:

Նույն կարգով lock; std::lock/scoped_lock (atomic); try_lock timeout:

std::lock ի՞նչ է անում:

Մի քանի mutex lock է անում atomic, deadlock-free ալգորիթով:

scoped_lock (C++17):

Մի քանի mutex, RAII, deadlock-free (std::lock + lock_guard միասին):

Circular wait ինչպես կտրել:

Global կարգ սահմանել mutex-ների համար, միշտ այդ կարգով lock:

Ամփոփում (Cheat Sheet)

DEADLOCK = երկու+ thread սպասում են իրար անվերջորեն (կախված)

ԻՆՉՈՒ: տարբեր կարգով մի քանի lock -> circular wait

4 ՊԱՅՄԱՆ (Coffman, բոլոր 4 պետք են):

Mutual Exclusion | Hold and Wait | No Preemption | Circular Wait

ԼՈՒԾՈՒՄ:

1. ՆՈՒՅՆ ԿԱՐԳՈՎ lock (ամենապարզ)
2. `std::lock` / `std::scoped_lock` (C++17, atomic մի քանի)
3. `try_lock` timeout-ով
4. քիչ mutex, փոքր critical section

ԿԱՆՈՆ: global lock ordering -> circular wait կոտրվում